

DAEDALUS

Plugin Runtime — Architecture & SPI Reference

:: Plugin lifecycle, JAR discovery, classloader isolation, event bus ::

Module: daedalus-plugin-runtime

Purpose: Embeddable plugin system for generators, solvers, and extensions.

Key guarantees: Per-plugin failure isolation • Dependency-ordered boot • Classloader leak prevention • Spring-free SPI

1. Service Provider Interface (SPI)

The entire plugin system is built around the **MazePlugin** interface. Plugins are discovered via **ServiceLoader** (both classpath and external JARs). The SPI is deliberately Spring-free so it can be embedded in any host.

1.1 MazePlugin Interface (Core Methods)

```
public interface MazePlugin {
    PluginManifest manifest();                // Required – id, displayName, version, requires[]
    default void init(PluginContext ctx) {}  // Stash ctx if needed
    default void registerAlgorithms(PluginContext ctx) {}
    default void start(PluginContext ctx) {}
    default void stop(PluginContext ctx) {}
    default List<AlgorithmDescriptor> contributedAlgorithms() { return List.of(); }
}
```

1.2 PluginManifest (Compact Constructor + Null Guards)

The record enforces non-null **id**, **displayName**, **version**. **requires** is normalized to empty array if null. This prevents NPEs in dependency sorting.

```
public record PluginManifest(
    String id, String displayName, String version,
    String author, String description, String[] requires
) {
    public PluginManifest { ... Objects.requireNonNull(...) ... }
}
```

Test: `PluginManifestNullGuardTest` — verifies all null-rejection and normalization behavior.

2. Lifecycle State Machine

State	Meaning	Transition
DISCOVERED	Loaded via ServiceLoader or JAR scan	→ INITIALIZED (on init success)
INITIALIZED	init() completed	→ REGISTERED
REGISTERED	registerAlgorithms() completed	→ STARTED
STARTED	start() completed — fully live	→ STOPPED (on shutdown)
STOPPED	stop() completed cleanly	—
FAILED	Any phase threw	Terminal — error recorded

bootAll() iterates in dependency order (topological sort on **requires**). Failures are isolated — one plugin crashing does not stop others. **shutdownAll()** only calls stop() on STARTED plugins and always closes external classloaders.

2.1 Dependency Ordering Example

If plugin "child" declares **requires: ["parent"]**, then parent reaches STARTED before child even receives init().

3. PluginManager — Discovery & Orchestration

The heart of the runtime. Two main entry points: **discover()** and **bootAll()**.

3.1 discover()

```
// 1. Built-in plugins (classpath ServiceLoader)
for (MazePlugin p : ServiceLoader.load(MazePlugin.class)) { register(p, "classpath"); }

// 2. External JARs
Files.list(pluginDir).filter(jar -> jar.endsWith(".jar"))
    .forEach(jar -> {
        URLClassLoader cl = new URLClassLoader(new URL[]{jar.toUri().toURL()}, parent);
        externalLoaders.add(cl);
        ServiceLoader.load(MazePlugin.class, cl).forEach(p -> register(p, jar.getFileName()));
    });
```

3.2 bootAll() — with per-plugin error isolation

```
for (Entry e : registry.sortedByDependencies()) {
    try {
        e.plugin().init(ctx);      registry.advance(id, INITIALIZED);
        e.plugin().registerAlgorithms(ctx); registry.advance(id, REGISTERED);
        e.plugin().start(ctx);     registry.advance(id, STARTED);
    } catch (Throwable t) {
        registry.fail(id, t);
        spring.publishEvent(new PluginFailedEvent(this, id, version, phase, t));
    }
}
```

3.3 shutdownAll() — classloader leak prevention (audit fix)

Every external JAR gets its own **URLClassLoader**. On shutdown (or reload), **shutdownAll()** calls `close()` on every tracked loader and clears the list. This prevents file-handle leaks on Windows and allows clean re-discovery after plugin directory changes.

Test: `PluginManagerLifecycleTest.shutdownAll_closesAndClearsExternalClassloaders` + `PluginManagerJarDiscoveryTest`

4. Event Bus (Spring-free POJOs)

All events extend **PluginEvent** (pure POJO with source + timestamp). Under Spring they are published via `ApplicationEventPublisher` and received with `@EventListener`. The STOMP bridge (`MazeWebSocketController`) listens for `PluginFailedEvent` and forwards it as `PluginFailedFrame` to `/topic/plugins/failures`.

Event	When Fired	Consumers
PluginFailedEvent	Any lifecycle phase throws (INIT/REGISTER/START/STOP/UNREGISTER)	STOMP bridge, monitoring, alerts
MazeGeneratedEvent	MazeGenerationService finishes	Plugins that want to react to new mazes
MazeSolvedEvent	Solver completes a run	Leaderboard, replay, analytics plugins
PlayerMovedEvent	Player makes a valid move in a session	Real-time UI sync, achievement plugins

5. Test Coverage (Locked by Audit)

Two comprehensive test classes exercise the entire subsystem without requiring a full Spring Boot context.

Test Class	Focus	Key Assertions
PluginManagerJarDiscoveryTest	Real JAR + ServiceLoader discovery	JAR loaded, classloader tracked, bootAll reaches STARTED, ...
PluginManagerLifecycleTest	Lifecycle ordering, failure isolation, events, ...	Dependency order, per-plugin isolation, PluginFailedEvent pub...
PluginManifestNullGuardTest	SPI contract	Null id/displayName/version rejected, requires normalized to []

SamplePlugin (test fixture) is packaged into a real JAR at test time to prove end-to-end JAR discovery works exactly as production external plugins will.

This document was generated from the full plugin-runtime source + tests. The frontend TypeScript equivalents for PluginFailedFrame / PluginEvent payloads are in daedalus-api-dtos.ts.